On Task Type Diversity

Document Number:	D4050R0
Date:	2026-03-11
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. Coroutine Types in Production

- 2.1 The Allocator Ceremony
- 2.2 Two Empty Structs
- 2.3 Library Upgrade Breaks Downstream
- 2.4 Adding a Default Does Not Prevent Fragmentation

3. Why the Invariants Differ

- 3.1 Frame Allocator
- 3.2 Foreign Awaitables
- 3.3 Timing
- 3.4 Symmetric Transfer

4. The `IoAwaitable` Protocol

- 4.1 Closed by Design
- 4.2 Contrast with `task`

5. Cross-Domain Bridges

- 5.1 Sender Bridges
- 5.2 The Bridge Pattern

6. No One True Task

Acknowledgements

- References
  - WG21 Papers

Talks

Other

Abstract

Seven coroutine libraries independently converged on a one-parameter task type. Each enforces domain-specific safety invariants through the promise. The only production two-parameter coroutine type - Asio's `awaitable<T, Executor>` - provides a type-erased default; users who opt out for performance immediately hit type incompatibility. `std::execution::task` (P3552R3[1]) adds a second template parameter with no default at all. This paper documents why the invariants differ by design and recommends that `std::execution::task` be scoped to the sender domain.

This paper is one of a suite of five. The companion papers are D4053R0[12], "Sender I/O: A Constructed Comparison"; D4054R0[13], "Two Error Models"; D4055R0[14], "Consuming Senders from Coroutine-Native Code"; and D4056R0[15], "Producing Senders from Coroutine-Native Code."

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The authors developed and maintain Corosio[3] and Capy[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The frame allocator gap (Section 3) was identified by Peter Dimov. The cross-library bridges (Section 5) were authored by Klemens Morgenstern. Neither is a co-author. The authors provide information, ask nothing, and serve at the pleasure of the chair.

---

2. Coroutine Types in Production

folly::coro and Boost.Cobalt are in production at scale. cppcoro was deployed at Facebook before folly::coro replaced it. Asio's `awaitable` is included as the only production two-parameter coroutine type.

Library	Declaration	Params	Source
asyncpp	`template<class T> class task`	1	task.hpp
Boost.Cobalt	`template<class T> class task`	1	task.hpp
Capy	`template<class T = void> struct task`	1	task.hpp
cppcoro	`template<typename T> class task`	1	task.hpp
aiopp	`template<typename Result> class Task`	1	task.hpp
libcoro	`template<typename return_type> class task`	1	task.hpp
folly::coro	`template<typename T> class Task`	1	Task.h
Boost.Asio	`template<class T, class Executor = any_io_executor> class awaitable`	2	awaitable.hpp
P3552R3 (std)	`template<class T, class Environment> class task`	2	task.hpp

The interface converged. The machinery diverged. Cobalt embeds intrusive list nodes for cancellation. folly::coro integrates with Facebook's fiber scheduler. Asio's `awaitable` is coupled to `io_context`. Capy propagates `io_env` through `await_suspend(h, io_env const*)`, delivering executor, stop token, and frame allocator at the suspension point.

Asio's `awaitable` provides a type-erased default (`any_io_executor`). Users who replace it with `io_context::executor_type` for performance immediately hit type incompatibility: `await_transform` rejects the mismatch.[26]

P3552R3[1] adds a second template parameter: `Environment`. No default. The parameter exists because `task` is both a coroutine return type and a sender. Every function that returns `task` is coupled to the sender environment model.

As of P3552R3[1], the wording declares `template<class T, class Environment> class task` with no default for `Environment` ([execution.syn][16], [task.class][16]). The design section states that "any empty type provides the default behaviour" but does not name or standardize such a type. The paper's own examples use `task<int>` with one parameter - ill-formed against its own wording. A future revision may add a default. Even so, the fragmentation argument in Section 2.4 holds: the `Environment` parameter exists so that users provide non-default environments, and the moment they do, the types diverge permanently.

Generic code that accepts `task<T>` does not accept `task<T, MyEnvironment>`. The moment any library in the dependency chain provides a non-default `Environment`, every function in the chain must agree on the type:

```
void spawn_work(std::execution::task<void> work);

std::execution::task<void, net_env> do_network();

spawn_work(do_network());  // does not compile:
                           // task<void, net_env> is not task<void>
```

Both are "the standard task." They do not compile together. No shipped code outside the reference implementation uses a non-default `Environment`. The parameter solves a problem that seven independent libraries solved without it.

One-parameter task types compose by construction. Two-parameter task types fragment by default.

2.1 The Allocator Ceremony

P3552R3[1] Section 4.6 shows how to enable allocator awareness. The user must define a custom environment struct and use it in every function signature:

```
struct alloc_env {
    using allocator_type =
        std::pmr::polymorphic_allocator<std::byte>;
};

std::execution::task<std::string, alloc_env>
read_body(tcp_socket& sock);

std::execution::task<json, alloc_env>
parse(std::string_view body);

std::execution::task<void, alloc_env>
write_response(tcp_socket& sock, std::string_view body);
```

Every production library:

```
task<std::string> read_body(tcp_socket& sock);
task<json>        parse(std::string_view body);
task<void>        write_response(tcp_socket& sock,
                                 std::string_view body);
```

No standard allocator-aware environment type is provided. Two users who independently define semantically identical structs produce incompatible task types.

2.2 Two Empty Structs

Two libraries define empty environments. Identical behavior. Incompatible types:

```
// Library A
struct env_a {};
std::execution::task<int, env_a> compute();

// Library B
struct env_b {};
std::execution::task<int, env_b> fetch();

// Both environments are empty. Both produce identical
// default behavior. The types are incompatible.
```

The standard task type fragments on contact with itself.

2.3 Library Upgrade Breaks Downstream

A library that upgrades from the default environment to a custom one changes the return type and breaks every downstream caller:

```
// v1.0: default environment
std::execution::task<response> http_get(url_view);

// v2.0: needs allocator awareness
std::execution::task<response, http_env> http_get(url_view);

// Every downstream caller has a type mismatch.
```

Networking can use thread-local propagation for frame allocators. GPU can use device memory APIs. HFT can enforce zero allocation at compile time. Solutions exist when the promise serves its domain.

2.4 Adding a Default Does Not Prevent Fragmentation

A default delays fragmentation; it does not prevent it. The `Environment` parameter exists so that users provide non-default environments. The moment they do, fragmentation begins - and it is viral: every function in the call chain must agree on the type.

Asio demonstrates the mild case. Asio's second parameter is an executor - a narrow concept with a fixed interface (`execute(f)`). Type erasure works: `any_io_executor` wraps any executor behind virtual dispatch. The default covers most users. Even so, users who replace `any_io_executor` with `io_context::executor_type` for performance immediately hit type incompatibility: `await_transform` rejects the mismatch[26], cross-context `co_await` fails at the

type level[27], and custom awaitables cannot compose with Asio's `awaitable` without matching the executor type[28]. The friction is documented across multiple StackOverflow questions. This is the mild case because the escape hatch exists - users can always fall back to `any_io_executor` at the cost of one virtual dispatch per operation.

P3552R3[1]'s `Environment` is the severe case. The environment is not a narrow concept with a fixed interface. It is an open query-response protocol: `get_stop_token`, `get_scheduler`, `get_allocator`, and any user-defined query, each with a different return type. The set of queries is unbounded. A type-erased `any_environment` would require `std::any`-style dynamic dispatch for every query, each with a different return type. This is technically possible. It destroys the compile-time composition that is the sender model's entire value proposition - completion signatures become runtime-dependent, `then` and `let_value` cannot deduce return types, and the zero-overhead property that justifies the model's complexity is gone. An `any_environment` that preserves the sender model's compile-time properties does not exist and cannot exist, because the query set is open-ended. The fragmentation is not a performance trade-off with an escape hatch. It is a choice between fragmentation and abandoning the properties that make the sender model worth having.

The diversity in the table above is not fragmentation. It is fitness.

---

3. Why the Invariants Differ

Every production library extends the C++20 awaitable protocol with domain-specific safety invariants. P3552R3[1]'s `await_transform` + `affine_on` maintains executor continuity and stop token propagation for recognized senders within `std::execution` (P2300R10[9]). Four gaps remain. Sections 3.1, 3.2, and 3.4 are engineering gaps with papers in progress - they may be closed in future revisions. Section 3.3 is architectural: the sender model's deferred delivery and the networking model's immediate delivery are contradictory requirements that no single task type can satisfy.

3.1 Frame Allocator

`affine_on` returns the coroutine to the correct scheduler thread. The thread-local frame allocator on that thread is whatever the previous coroutine left - not the allocator for this chain. No hook exists between `affine_on` completing and the coroutine body resuming where TLS can be restored. Peter Dimov identified this gap. P3980R0[6] addresses allocator sourcing but does not restore TLS on resumption.

3.2 Foreign Awaitables

P3552R3[1]'s `await_transform` recognizes senders. A non-sender awaitable - `cppcoro::task<int>`, a Cobalt awaitable, any C++20 awaitable - passes through without `affine_on`, without stop token wiring, without environment propagation. All three invariants are lost. Networking libraries reject non-conforming awaitables at compile time. P3552R3[1] permits them. Non-sender domains require their own task types.

3.3 Timing

Senders deliver context through `connect` / `start` after the coroutine frame is allocated. Networking delivers context at `co_await` time through `await_suspend`. The frame allocator must be available before the child's `operator new` ([dcl.fct.def.coroutine][16]) - before any sender machinery runs. No mechanism in `std::execution` (P2300R10[9]) delivers an allocator to `operator new` without `allocator_arg_t`. Deferred delivery serves senders. Immediate delivery serves networking. The requirements are incompatible.

3.4 Symmetric Transfer

The sender-awaitable's `await_suspend` returns `void`, foreclosing symmetric transfer. Jonathan Müller confirmed in P3801R0[7]: ***"a thorough fix is non-trivial and requires support for guaranteed tail calls."*** Dietmar Kühl documented the gap in P3796R1[8].

---

4. The `IoAwaitable` Protocol

`IoAwaitable` is not proposed for standardization. It is the authors' design and is presented here as one example of what domain-specific compile-time enforcement looks like - not as the only possible design. The cross-domain bridges in Section 5, authored independently by Klemens Morgenstern, provide the stronger evidence that domain-specific protocols are the norm. P4003R0[2] defines the concept:

```
template<typename A>
concept IoAwaitable =
    requires(A a, std::coroutine_handle<> h,
             io_env const* env)
    {
        a.await_suspend(h, env);
    };
```

The `io_env` pointer carries three fields:

```
struct io_env
{
    executor_ref executor;
    std::stop_token stop_token;
    std::pmr::memory_resource* frame_allocator;
};
```

These are the three invariants Section 3 identifies as gaps in P3552R3[1]. `IoAwaitable` delivers all three at `await_suspend` time. Every awaitable in the chain receives the environment. No deferred delivery. No timing gap.

4.1 Closed by Design

The promise's `await_transform` passes `io_env*` to `await_suspend(h, env)`. Any awaitable that does not accept the second parameter fails to compile. The check applies to all awaitables - coroutine-backed or not. A foreign awaitable with the standard `await_suspend(coroutine_handle<>)` signature does not compile. The invariant violation is a type error, not a silent runtime degradation.

4.2 Contrast with `task`

P3552R3[1]'s `await_transform` calls `as_awaitable(expr, promise)`. For recognized senders, this wraps them in an awaiter that applies `affine_on` and stop token propagation. For non-sender awaitables, the expression passes through unchanged - no `affine_on`, no stop token, no environment propagation. The `Environment` template parameter controls what the promise advertises through `get_env()`, but the awaitable never sees it unless it is a sender that goes through `connect` / `start`.

Property	`IoAwaitable`	P3552R3 `task`
Foreign awaitable `co_await`ed	Compile error	Compiles, invariants lost
Non-coroutine awaitable gets environment	Yes (`io_env*` in `await_suspend`)	No (only senders via `connect`)
Enforcement point	`co_await` site (compile time)	`connect` / `start` (runtime path)
Invariant violation mode	Type error	Silent

`IoAwaitable` demonstrates that the `await_suspend` signature is the natural enforcement point, and it works for all awaitables, not just coroutine-backed ones. P3552R3[1]'s permissive model is appropriate for the sender domain. It cannot serve as a general-purpose correctness boundary because it has no mechanism to reject non-conforming awaitables.

---

5. Cross-Domain Bridges

The cross_await[17] repository (Klemens Morgenstern) contains four cross-library composition examples. All four require explicit adaptor code.

Outer library	Inner library	Adaptor file	Lines
Cobalt	Capy	`cobalt_capy.cpp`	67
TMC	Capy	`tmc_capy.cpp`	51
Capy	Cobalt	`corosio_cobalt.cpp`	105
Capy	TMC	`corosio_tmc.cpp`	60

From `cobalt_capy.cpp`:

```
template<typename T>
struct capy_task_adaptor {
    capy::task<T> tt;
    capy::io_env env;
    std::stop_source src;
    boost::asio::cancellation_slot slt;

    template<typename Promise>
    auto await_suspend(std::coroutine_handle<Promise> h) {
        auto& p = h.promise();
        env.executor        = p.get_executor();
        env.stop_token      = src.get_token();
        env.frame_allocator = p.get_allocator().resource();
        slt = p.get_cancellation_slot();
        slt.assign([this](auto ct) {
            if ((ct & boost::asio::cancellation_type::terminal) !=
                      boost::asio::cancellation_type::none)
                src.request_stop();
        });
        return tt.await_suspend(h, &env);
    }
};
```

The bridge knows both protocols. It is specific to the pair. There is no generic bridge because the invariants are domain-specific.

5.1 Sender Bridges

D4055R0[14]: consuming senders from `IoAwaitable` code. One class template. Satisfies `IoAwaitable`, propagates stop token, dispatches back to the originating executor. `set_error(std::error_code)` becomes `io_result<T>` - no exceptions. Does not use `std::execution::task`.

D4056R0[15]: producing senders from `IoAwaitable` code. Compound I/O results are rejected at compile time. The coroutine body inspects the compound result, reduces it to `error_code`, and the bridge routes it through three channels without exceptions.

5.2 The Bridge Pattern

All six bridges share three properties:

- **Opt-in.** No bridge is automatic.
- **Explicit.** The bridge code is visible. No silent invariant translation.
- **Responsibility.** The bridge author maintains invariants across the boundary.

Cross-domain composition works. The C++20 awaitable protocol is the surface all bridges bottom out to.

---

6. No One True Task

Gor Nishanov stated the design intent in P1362R0[10] (Section 3.7):

> *"The separation is based on the observation that coroutine types and awaitables could be developed independently of each other by different library vendors. In a product, one could use N coroutine types that describe how coroutine behaves and M awaitables describing how a particular asynchronous API should be consumed. Users can freely mix and match coroutines and awaitables as needed."*

And at CppNow 2017[11]: "We did not want to tie it to a particular task library. [...] We wanted them to be open."

`promise_type` is the extension point. Unifying all task types into one forces either carrying every domain's invariants (bloat) or carrying none (unsafe). D4053R0[12] and D4054R0[13] document a concrete example: I/O returns `(error_code, size_t)`, and the three-channel model forces a routing decision that loses the byte count, the channel semantics, or both.

Acknowledgements

The author thanks Gor Nishanov for the coroutine model's explicit support for task type diversity; Peter Dimov for identifying the frame allocator propagation gap; Klemens Morgenstern for Boost.Cobalt and the cross_await bridges; Dietmar Kühl for P3552R3, P3796R1, and `beman::execution`; Jonathan Müller for confirming the symmetric transfer gap in P3801R0; Steve Gerbino for the constructed comparison and bridge implementations; and Mungo Gill, Mohammad Nejati, and Michael Vandeberg for feedback.

References

WG21 Papers

1. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
2. P4003R0 - "Coroutines for I/O" (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). https://wg21.link/p4003r0
3. P4007R1 - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r1
4. P4014R0 - "The Sender Sub-Language" (Vinnie Falco, 2026). https://wg21.link/p4014r0

5. P2583R1 - "Symmetric Transfer and Sender Composition" (Mungo Gill, Vinnie Falco, 2026). https://wg21.link/p2583r1
6. P3980R0 - "Task's Allocator Use" (Dietmar Kühl, 2026). https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3980r0.html
7. P3801R0 - "Concerns about the design of `std::execution::task`" (Jonathan Müller, 2025). https://wg21.link/p3801r0
8. P3796R1 - "Coroutine Task Issues" (Dietmar Kühl, 2025). https://wg21.link/p3796r1
9. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
10. P1362R0 - "Incremental Approach: Coroutine TS + Core Coroutines" (Gor Nishanov, 2018). https://wg21.link/p1362r0
11. D4053R0 - "Sender I/O: A Constructed Comparison" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4053r0
12. D4054R0 - "Two Error Models" (Vinnie Falco, 2026). https://wg21.link/d4054r0
13. D4055R0 - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4055r0
14. D4056R0 - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4056r0

Talks

11. Gor Nishanov, "C++17 coroutines for app and library developers," CppNow 2017. https://www.youtube.com/watch?v=proxLbvHGEQ

Other

16. C++ Working Draft - (Richard Smith, ed.). https://eel.is/c++draft/
17. Klemens Morgenstern, cross_await - "co_await one coroutine library from another" (2026). https://github.com/klemens-morgenstern/cross_await
18. cppcoro - C++ coroutine library (Lewis Baker). https://github.com/lewissbaker/cppcoro
19. libcoro - C++20 coroutine library (Josh Baldwin). https://github.com/jbaldwin/libcoro
20. asyncpp - Async coroutine library (Péter Kardos). https://github.com/petiaccja/asyncpp
21. aiopp - Async I/O library (Joel Schumacher). https://github.com/pfirsich/aiopp
22. beman::task - P3552R3 reference implementation. https://github.com/bemanproject/task
23. folly::coro - Facebook coroutine task type. https://github.com/facebook/folly/blob/main/folly/experimental/coro/Task.h
24. Boost.Cobalt - Boost coroutine task type (Klemens Morgenstern). https://github.com/boostorg/cobalt/blob/develop/include/boost/cobalt/task.hpp
25. Capy - Coroutine primitives library (Vinnie Falco). https://github.com/cppalliance/capy
26. Boost asio using concrete executor type with c++20 coroutines causes compilation errors - StackOverflow (2024). https://stackoverflow.com/questions/79115751
27. Can I co_await an operation executed by one io_context in a coroutine executed by another in Asio? - StackOverflow (2022). https://stackoverflow.com/questions/73517163

28. How to create custom awaitable functions that can be called with co_await inside asio::awaitable functions - GitHub asio#795 (2022). https://github.com/chriskohlhoff/asio/issues/795